



ENSEEIH

SYSTÈMES CONCURRENTS ET COMMUNICANTS
RAPPORT

Projet HagiMule

Élèves :
Leïlie CANILLAC
Vianney HERVY

Enseignant :
Daniel HAGIMONT

15 janvier 2025

Table des matières

1	Introduction	1
2	Approches de la fragmentation et du parallélisme	1
3	Architecture globale	2
3.1	Annuaire	2
3.2	Client	2
4	Lancement en ligne de commande	3
5	Interfaces graphiques	4
5.1	Interface graphique serveur annuaire	4
5.2	Interface graphique client	4
6	Synchronisation annuaire / démon	5
6.1	Connexion persistante	5
6.2	Fichiers servables	5
6.3	Obstacles rencontrés	5
7	Code	6
7.1	Dépendances	6
8	Exigences non satisfaites	6
9	Améliorations possibles	6
10	Annexes	6

Table des figures

1	Diagramme de classes de l'annuaire	3
2	Documentation du lancement du serveur annuaire	3
3	Documentation du lancement du client	4
4	Interface graphique du serveur annuaire	4
5	Interface graphique du client	5
6	Diagramme de classes global simplifié	7

1 Introduction

Le projet contient les éléments principaux suivants :

- un annuaire qui enregistre les fichiers et les informations des fichiers des clients connectés,
- Un client qui permet de se connecter à un annuaire, de servir des fichiers et de les télécharger.

2 Approches de la fragmentation et du parallélisme

Notre approche initiale était de découper le fichier demandé en fragments de taille fixe, puis de distribuer ces fragments aux hôtes possédant le fichier, quitte à ce que plusieurs hôtes

servent plusieurs fragments différents. Pour cela, il fallait créer une connection socket pour chaque fragment.

Notre approche finale est de découper le fichier en autant de fragments que d'hôtes possédant le fichier, puis de demander un fragment à chaque hôte. Ainsi, une seule connection socket TCP par hôte est nécessaire.

3 Architecture globale

Le projet est réparti en plusieurs packages :

- `fr.n7.hagimule` : contient les classes utilitaires
- `fr.n7.hagimule.client` : contient les classes client
- `fr.n7.hagimule.client.daemon` : contient les classes relatives au démon
- `fr.n7.hagimule.client.downloader` : contient les classes relatives au téléchargeur
- `fr.n7.hagimule.client.gui` : contient les classes relatives à l'interface graphique du client
- `fr.n7.hagimule.diary` : contient les classes relatives à l'annuaire
- `fr.n7.hagimule.test` : contient les tests unitaires

3.1 Annuaire

Un annuaire est accessible via RMI¹. Il lui faut donc un "serveur" qui réalise seulement le binding de l'annuaire. C'est la classe `DiaryServer`. L'annuaire lui-même est implémenté dans la classe `DiaryImpl`.

Lors des premières itérations, nous avons conçu l'annuaire de telle sorte à ce que toutes ses méthodes soient publiques. Après plusieurs réécritures et changements d'implémentation interne, nous avons décidé de spécifier clairement les méthodes dans une interface. Plus tard encore, avec l'apparition des connections persistantes, nous avons réalisé que certaines méthodes de l'annuaire devaient être uniquement accessibles par un client ou uniquement par un serveur. Nous avons donc créé deux interfaces : `ClientDiary` et `ServerDiary`. Elles spécifient chacune un annuaire tel qu'il est vu par un client ou un serveur. Le diagramme de classes est visible sur la figure 6.

3.2 Client

Un client (classe `Client`) a un démon (classe `Daemon`) et un téléchargeur (classe `Downloader`).

Un démon est toujours à l'écoute d'une connection TCP. Lorsqu'une connection est établie, il crée un esclave (classe `DaemonSlave`) qui va servir le fragment demandé. Cet esclave s'exécute dans un thread séparé et ne bloque donc pas le démon qui reste à l'écoute.

Un téléchargeur est un objet simple qui ne s'exécute pas dans un thread séparé. Sa méthode `downloadFile(String)` crée une tâche de téléchargement (classe `DownloadTask`) qui s'exécute dans un thread séparé. Cette classe de tâche gère l'ensemble de la logique pour le téléchargement d'un fichier. Elle récupère hôtes et crée les esclaves (classe `DownloaderSlave`) qui vont télécharger les fragments.

Un esclave de téléchargement est créé avec un hôte et des informations sur le fragment à télécharger. Il se connecte à l'hôte, lui demande le fragment, le reçoit et l'écrit lui-même dans l'attribut (synchronized) `fileBytes` de sa tâche parente avec la méthode `writeFragment(byte[], int)`. Ainsi, la tâche de téléchargement n'est pas responsable de l'assemblage des fragments. Elle attend juste la terminaison de tous ses esclaves avant d'écrire le fichier sur le disque.

1. https://en.wikipedia.org/wiki/Java_remote_method_invocation

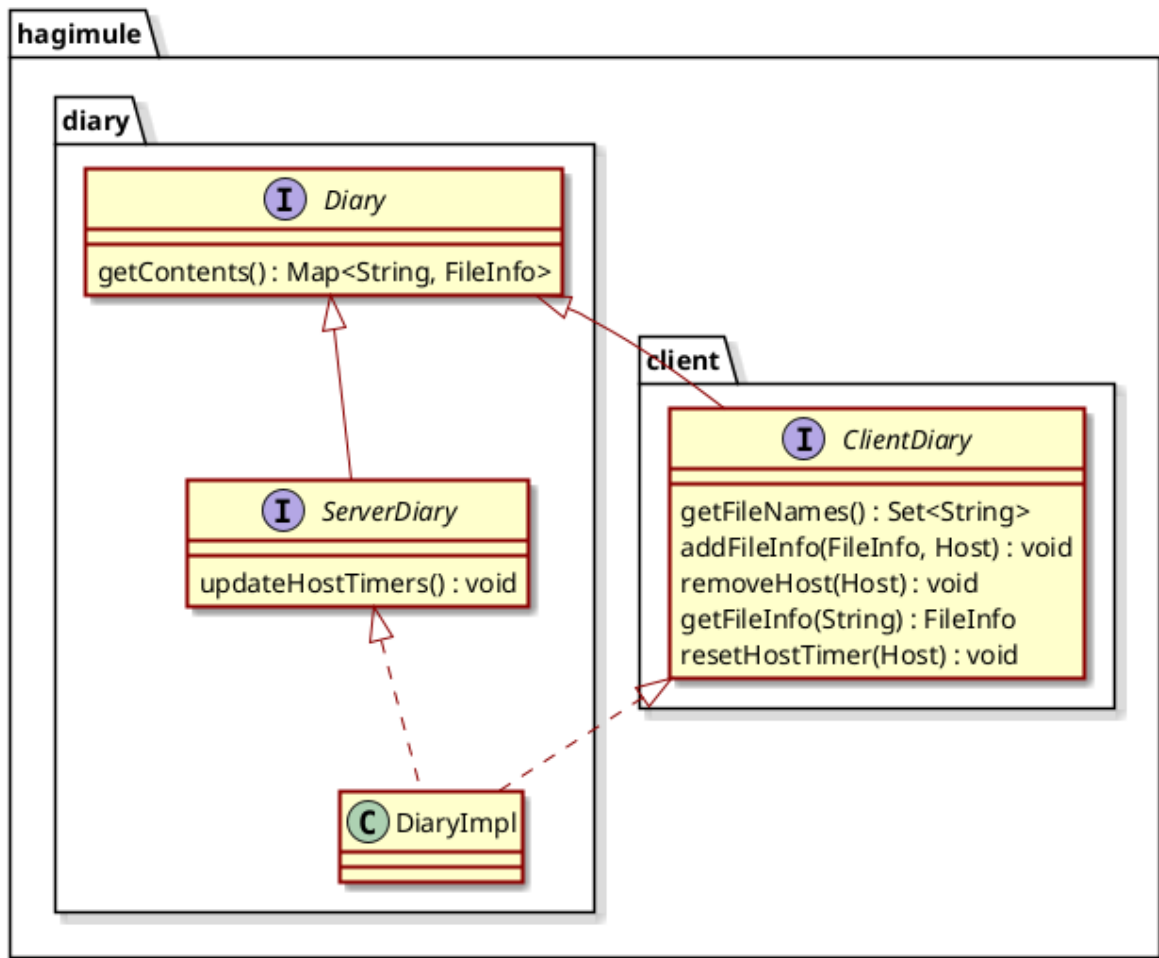


FIGURE 1 – Diagramme de classes de l'annuaire

4 Lancement en ligne de commande

Un client et un serveur annuaire peuvent être lancés en ligne de commande (l'hébergeur peut vouloir fonctionner uniquement en mode démon). On peut spécifier des options en argument comme décrit dans les figures 2 et 3.

Lorsqu'un client est lancé, il va de suite essayer de se connecter à l'annuaire. Si l'annuaire n'est pas accessible, il réessaie en boucle toutes les 10 secondes. Dès que la connexion est établie, il le signale à l'utilisateur et synchronise les fichiers servables.

```

usage: ./scripts/diary [--gui]
-g,--gui    Run the server with a GUI
-h,--help   Print this message
    
```

FIGURE 2 – Documentation du lancement du serveur annuaire

```
usage: ./scripts/client [-p <daemon port>] [-d <diary server IP>]
-d,--diary <arg>      IP address of the diary server
-g,--gui               Run the client with a GUI
-h,--help              Show help
-p,--port <arg>       Port number for the daemon
```

FIGURE 3 – Documentation du lancement du client

5 Interfaces graphiques

Les interfaces graphiques de ce projet ont été réalisées avec les packages `java.awt`² et `javax.swing`³. Seuls les deux éléments principaux : le client et le serveur annuaire ont une interface graphique.

5.1 Interface graphique serveur annuaire

L'interface graphique du serveur annuaire (figure 4) ne sert qu'à l'affichage des fichiers servables et des hôtes connectés, on ne peut pas interagir avec.

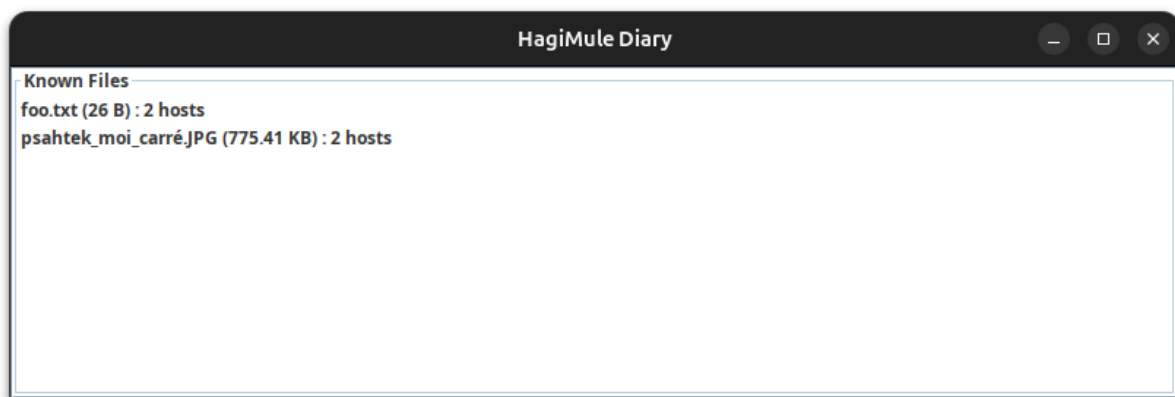


FIGURE 4 – Interface graphique du serveur annuaire

5.2 Interface graphique client

L'interface graphique du client (figure 5) a été réalisée assez tôt dans le projet car il était nécessaire d'interagir avec le client pour tester les fonctionnalités. Elle permet de choisir l'IP du serveur annuaire, de lister les fichiers servables et d'en télécharger un. Les téléchargements en cours sont affichés dans le rectangle inférieur.

2. https://en.wikipedia.org/wiki/Abstract_Window_Toolkit

3. [https://en.wikipedia.org/wiki/Swing_\(Java\)](https://en.wikipedia.org/wiki/Swing_(Java))

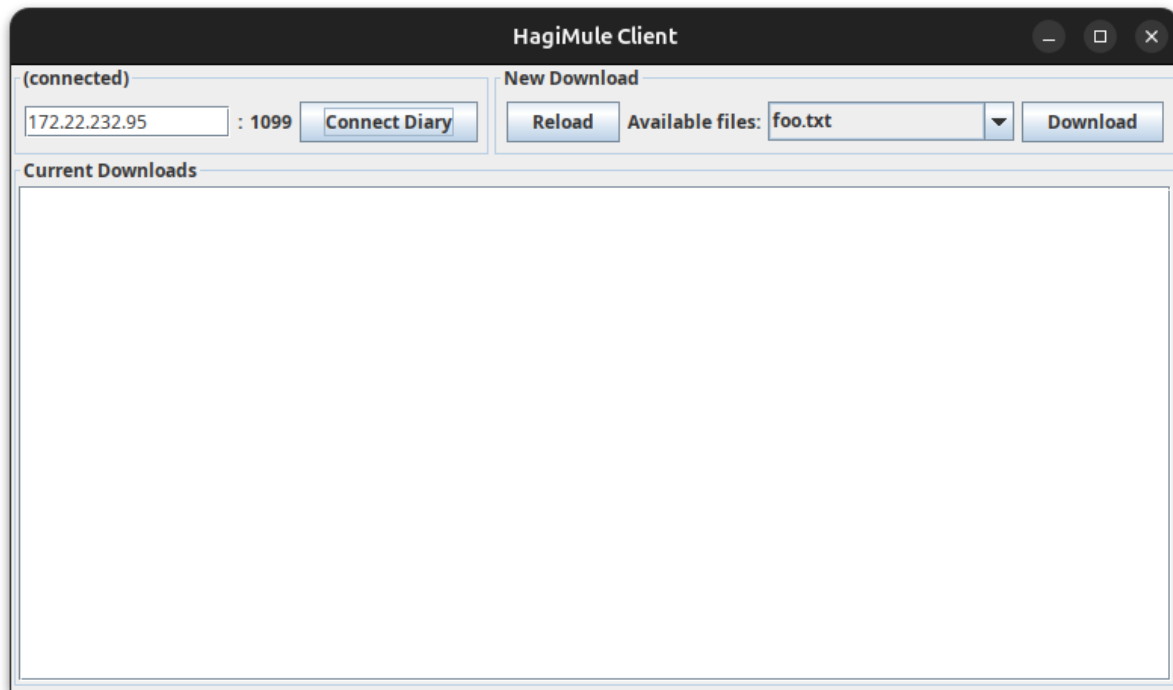


FIGURE 5 – Interface graphique du client

6 Synchronisation annuaire / démon

6.1 Connection persistante

Un des enjeux du projet est d'avoir un annuaire à jour. Pour cela, nous avons mis en place un système de connection dite "keep-alive" entre le démon et l'annuaire.

L'annuaire possède pour chaque hôte dont il a connaissance un timer. Chaque seconde, le serveur hébergeant l'annuaire décrémente ce timer. Si le timer atteint 0, l'annuaire considère l'hôte associé comme inaccessible et le supprime de sa liste.

Le démon quant à lui, a accès via RMI à une méthode `resetHostTimer(Host)` qui remet à 3 le timer de l'hôte passé en paramètre (lui-même). Nous avons fait le choix de créer un thread qui appelle cette méthode à chaque seconde plutôt que de l'appeler à chaque fois que le démon effectue une action.

6.2 Fichiers servables

Il n'est pas seulement nécessaire de synchroniser l'accessibilité des hôtes, mais aussi les fichiers qu'ils servent. Pour cela, nous avons implémenté une méthode de surveillance des fichiers servables avec l'aide du package `java.nio.file`. Cette surveillance effectue une synchronisation avec l'annuaire chaque fois qu'un fichier est ajouté, supprimé ou modifié.

6.3 Obstacles rencontrés

La conception des solutions de synchronisation était plutôt directe et simple, mais ce qui a posé problème était la gestion des accès concurrents à l'annuaire. La solution était donc de marquer les méthodes de l'annuaire comme `synchronized`.

7 Code

L'ensemble du code est compatible avec Java 11⁴.

Le code est en partie documenté suivant les conventions Javadoc⁵. Des commentaires sont présents lorsque le code n'est pas suffisamment clair.

Des tests ont été réalisés tout au long du développement. Vous trouverez ainsi des suites de tests JUnit⁶ dans le package `fr.n7.hagimule.test`. Ceux-ci n'ont pas été mis-à-jour en fin de développement et ne sont pas garantis de passer ni même de compiler.

7.1 Dépendances

Le projet utilise trois dépendances externes au JDK :

- `org.apache.commons.cli`⁷ pour la gestion des arguments de la ligne de commande lors du lancement d'un client ou du serveur annuaire,
- `org.junit`⁸ pour les tests unitaires,
- `org.hamcrest`⁹ pour les tests unitaires.

8 Exigences non satisfaites

- **Robustesse** : Si un hôte se déconnecte, supprime son fichier ou que le fragment se perd, le téléchargement échoue, il n'y a pas de tentative de reconnection.
- **Optimisation** : Le téléchargeur ne prend pas en compte l'occupation des sources lors de la répartition des fragments.
- **Compression** : Ni les fragments individuels, ni les fichiers ne sont compressés. Chaque octet est envoyé tel qu'écrit sur le disque de l'hôte.

9 Améliorations possibles

En plus des exigences non satisfaites, voici quelques améliorations possibles :

- **Robustesse** : Implémenter un système de clé sha256 pour différencier les fichiers servis, plutôt que de se baser sur le nom du fichier. Cela permettrait de vérifier l'intégrité d'un fichier téléchargé.
- **Optimisation** : Permettre l'écriture sur disque d'un fragment même lorsque le fichier complet n'a pas été téléchargé. Cela diminuerait l'utilisation de la mémoire vive en cas de téléchargement de gros fichiers.

10 Annexes

4. <https://openjdk.java.net/projects/jdk/11/>

5. <https://docs.oracle.com/javase/8/docs/technotes/tools/windows/javadoc.html>

6. <https://junit.org/>

7. <https://commons.apache.org/proper/commons-cli/>

8. <https://junit.org/>

9. <https://hamcrest.org/>

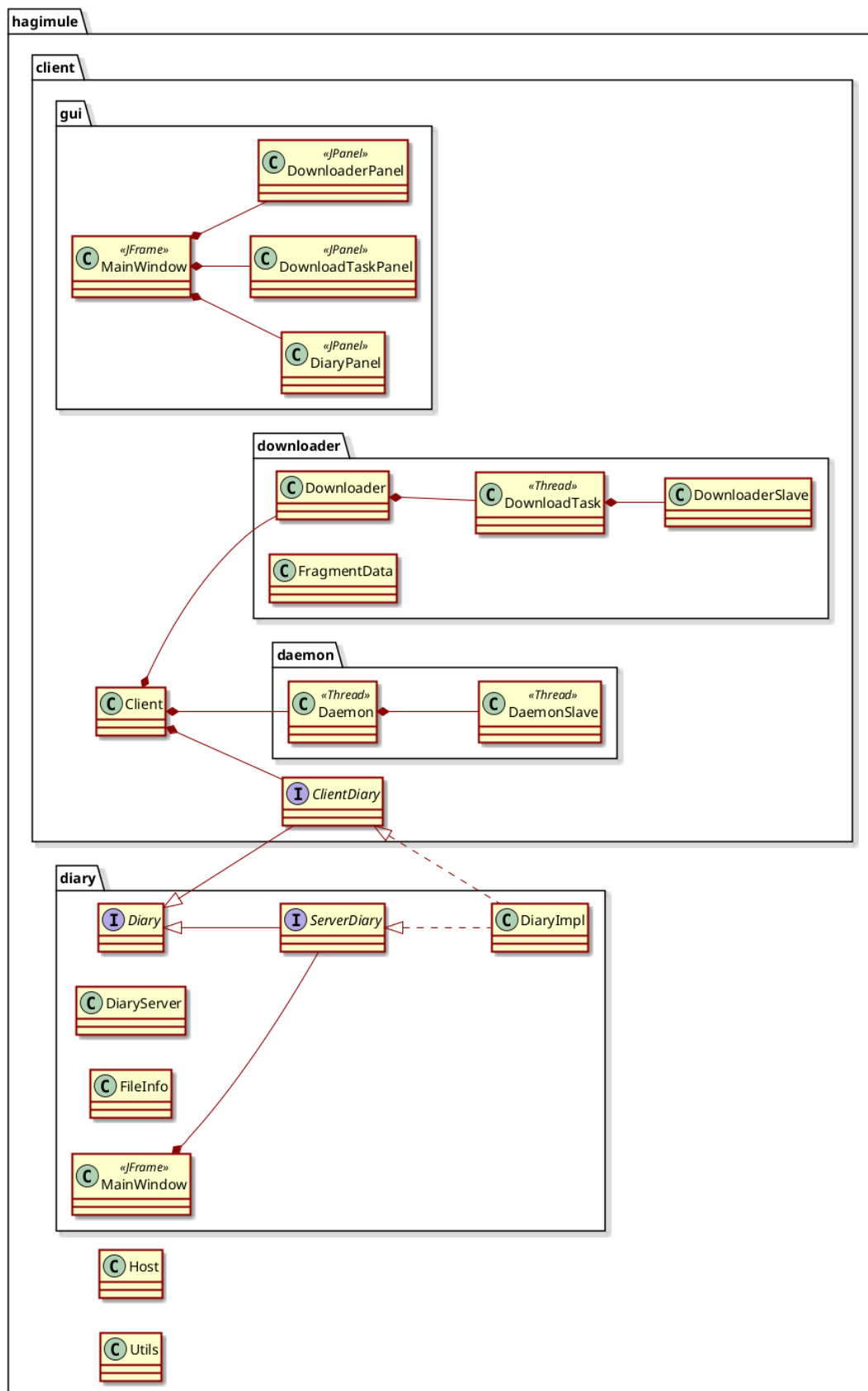


FIGURE 6 – Diagramme de classes global simplifié